1. Executive Summary: Problem, solution, and what your MVP actually Does.

Flight delays and cancellations disrupt millions of travelers every year, yet passengers typically receive little warning until disruptions are already underway. Whether it's a business trip with back-to-back meetings, a family vacation, or simply picking someone up from the airport, unexpected flight disruptions force last-minute decisions that could have been avoided with better information.

Flight Risk is a machine learning-powered flight predictability tool that estimates the probability of a flight being on time, experiencing a short delay (under 30 minutes), experiencing a long delay (over 30 minutes), or being cancelled. Rather than reacting to disruptions after the fact, users can consult Flight Risk before their trip and make smarter, more informed travel decisions.

The MVP is a Streamlit web application where a user enters a flight's origin airport, destination airport, airline, and travel date. The app runs the input through a trained classifier model, built on historical U.S. flight and weather data, and returns a predicted outcome category along with the probability of each outcome. The interface is simple for all users: no account required, no technical knowledge needed. A user gets a clear, actionable risk assessment in seconds.

2. User & Use Case: Clear persona and usage narrative.

Imagine you're an operations manager who flies several times a year for work and occasionally for personal travel. Your time is tightly scheduled; a delayed flight doesn't just mean waiting at the gate; it can mean missing a client meeting, scrambling to rebook a rental car, or leaving a family member stranded at arrivals.

You have a Monday morning flight from Phoenix (PHX) to Chicago (ORD) for a 10 AM client presentation. You're not a data analyst, but wonder, "Is this flight likely to go sideways?" The meeting cannot be rescheduled. The night before, you open Flight Risk, enter the route, airline, and departure date, and see that the model predicts a 61% probability of a long delay on that route.

Armed with this information, you book an earlier Sunday evening flight instead, arriving the night before and eliminating the risk entirely. Without Flight Risk, you would have had no reason to worry until the delay notification arrived at 5:45 AM.

Even if you're not a frequent flyer, Flight Risk can still help you avoid uncertainty when it comes to flights.

Imagine you're waiting to pick up a travelling family member at the airport: you still face the same uncertainty. Flight Risk gives you a heads-up if there's a high probability of delay, so you can leave later, avoid waiting in short-term parking, or adjust your own schedule accordingly.

3. System Design: High-level architecture diagram (can be ASCII or image

in repo); where the model sits, how data flows through.

High-Level Architecture

Historical Flight Data and Weather Features → data.py (Cleaning/Prep) → features.py (Feature Engineering) → train_model.py (Supervised Model Training) → artifacts/flight_risk_bundle.joblib → predictor.py (Interference and Route Ranking) → app.py (Streamlit User Interface) → Ranked Flight Options and Budget-Adjusted Recommendation

Where the Model Sits

The trained model is saved inside: Phase 3/ProjectCode/artifacts/flight_risk_bundle.joblib

→ joblib bundle includes:

- The scikit-learn preprocessing and classifier pipeline
- Label encoder
- Historical route/airline aggregate statistics and route options used by demo app
- Numeric fill values
- Feature column names

Data Flow

1. 'Train_model.py' loads a CSV or parquet dataset
2. 'data.py' cleans columns, creates labels, and performs a time-based train/validation/test split
3. 'features.py' adds: date, departure time, route, airline, weather, and historical aggregate features
4. 'modeling.py' trains a supervised 4-class classifier and saves metrics
5. . 'predictor.py' loads the trained bundle, finds historically common route options, builds interference features, predicts probability, and calculates "risk scores."
6. 'app.py' displays a Streamlit interface and adds a budget system for best value recommendation.

→ Added Budget Calculation:

expected_disruption_cost = short_delay_probability * short_delay_cost + long_delay_probability * long_delay_cost + cancellation_probability * cancellation_cost

risk_adjusted_cost = ticket_price + expected_disruption_cost

4. Data: Source(s), size, cleaning, splits.

Sources

→ The MVP uses historical commercial flight data and weather-reliant flight data stored locally in the project workspace.

→ Primary Modeling File: Flights_with_weather.parquet

→ Supporting/Local Files:

- ALL_FLIGHTS_2021_2023.csv
- flights_with_weather.csv
- flights_sample_2021_2023_100k.csv
- flights_sample_2021_2023_10k.csv
- Tornadoes_SPC_1950to2015.csv

→ Planned source for flight datasets is Kaggle. The weather-reliant parquet file is used for the best MVP because it includes origin and destination weather features

Size

ALL_FLIGHTS_2021_2023.csv:	3,540,975,418 bytes
flights_with_weather.csv:	858,860,089 bytes
flights_with_weather.parquet:	128,341,909 bytes
flights_sample_2021_2023_100k.csv:	20,507,378 bytes
flights_sample_2021_2023_10k.csv:	2,051,806 bytes
Tornadoes_SPC_1950to2015.csv:	5,144,997 bytes

→ The 'flights_with_weather.parquet' file contains 3,234,981 rows and 44 columns
→ The saved training run used these cleaned split sizes:
Train Rows: 34,761
Validation Rows: 39,099
Test Rows: 26,140

Cleaning

→ The cleaning pipeline is implemented in: /ProjectCode/src/flightrisk/data.py

1. Main Cleaning Steps:
   - Load CSV or Parquet input
   - Keep only selected base flight and weather columns
   - Convert flight date to datetime and cancellation, arrival delay, scheduled departure time, elapsed time, distance, and weather columns to numeric values
   - Drop rows missing required fields: flight date, airline code, origin/destination airport/scheduled departure time
   - Create a 4-class target label: 'on_time', 'short_delay', 'long_delay', 'cancelled'

Splits

→ Uses a time-based split when at least three years are present:
- Oldest training period (training set)
- Next year (validation set)
- Most recent year (test set)

→ If less than three years are given, the split is:
- 70% train
- 15% validation
- 15% test

DATA In GitHub

Large raw data is not in GitHub; the deployable repository includes the trained model artifact instead.

5. Models: What model(s) did you use: agent workflow on top of frontier models, supervised model(s), generative model(s), your own nanoGPT variant, or pre-trained models from elsewhere; include any fine-tuning, prompting, or workflow-design strategies.

Model Type

→ The MVP uses a supervised machine learning classifier trained with scikit-learn and predicts one of four classes:

1. ‘On_time’
2. ‘Short_delay’
3. ‘Long_delay’
4. ‘Cancelled’

Model Pipeline

The model pipeline is implemented in: /ProjectCode/src/flightrisk/modeling.py
This pipeline includes:

- Categorical preprocessing
    - Most frequent imputation
    - One-hot encoding with unknown category handling
- Numeric preprocessing
    - Median imputation
- Classifier
    - Logistic regression
    - ‘Class_weight = “balanced”’
    - ‘Max_iter = 500’

Features

Categorical Features:

- AIRLINE_CODE, ORIGIN, DEST
- Dep_time_bucket
- Month, day_of_week
- Is_weekend, is_holiday_window

Numeric Features:

- Dep_hour, dep_minute, CRS_ELASPED_TIME
- DISTANCE
- Airline_cancel_rate, airline_long_delay_rate
- route_cancel_rate, route_long_delay_rate
- origin_disruption _rate, dest_disruption_rate
- Route_month_delat_rate, route_month_cancel_rate
- Historical_option_support
- origin/destination weather columns

Weather Features (origin/destination airports):

- Temperature, pressure, humidity
- Wind speed, precipitation, and snowfall

Risk Score

After class probabilities are predicted, the app calculates: **risk_score** = cancelled + 0.6 * long_delay + 0.2 * short_delay

- This weights cancellations and long delays more heavily than short delays

Budget System

The budget system is a decision layer on top of the classifier output, which calculates: **risk_adjusted_cos**t = ticket_price + expected_disruption_cost

- This provides users with the cheapest option, safest option, and best value option

Generative AI/ nanoGPT Use

The current MVP does not include a deployed nanoGPT or external generative model. For this MVP, explanations are rule-based and generated from model outputs and supporting route or weather signals. Future work on the project could implement a generative explanation layer that summarizes why flights are ranked as they are.

6. Evaluation: Quantitative metrics where possible; qualitative assessment (e.g., user examples, error analysis).

To ensure predictions are realistic, our model is trained to use older flight data (2018-2022) and test how well it predicts future flights. We don't mix the data because realistically, you can't use tomorrow's weather to predict today's flight. This ensures that our model is actually learning patterns rather than just memorizing a specific list of flights.

A key part of evaluating our code is probability calibration, so we search for false negatives in our data- for example, when the app says a flight is safe but gets cancelled, the model has produced a false negative. This can cost a user time and money. By analyzing errors like this, we can ensure we are not missing key information and adjust our risk score outcome to produce a more accurate model.

Quantitatively, we also track the Cost-Benefit Ratio for the user. We calculate the difference between the ticket price of the "Lowest Risk" flight and the "Cheapest" flight. By multiplying this by the probability of a delay (P x Cost), we can provide a numeric value representing the "Expected Loss" for each travel option, allowing the user to make a data-driven financial decision. Then our model produces an outcome for predicted ticket price, risk-adjusted cost, budget gap, and if it falls within the budget.

7. Limitations & Risks: Failure modes, biases, data issues, privacy Concerns.

In statistical models, we need to distinguish the difference between predictable patterns and random events. A major limitation of our app is that we can't predict rare events, or high-impact events like glitches in airline data or ground stops due to security issues or plane malfunctions. These are all unpredictable events that fall outside of historical data. Even if our model shows 99% on time probability, there are always residual risks that can't be quantified by the data we have in our system.

Because we use older data patterns, there is a risk that we don't have imported data from updated flight rules. Route frequencies are way different now than they were back then, and if our model relies too heavily on older data sets, it might overestimate or underestimate the current reliability of airline information and create biased outcomes in the risk score.

Risks we encounter with the budget panel is over reliance from users. If a user sees a low-risk flight and decides to book a non-refundable hotel or a connection based on the data provided from flight risk, they are taking a gamble with the actual outcome. The app we created provides an estimate of uncertainty, not a guarantee of performance.

Privacy concerns would be another issue that would need to be analyzed. While our users are inputting flight information such as dates, destination, and financial constraints, this constitutes a digital footprint. There is a risk that the data is improperly stored or linked to a user profile and could be exploited for targeted price discrimination by third parties. To mitigate this,

we don't require a made account to use the app itself, and the panels are only used for local session calculations and are not stored in the training set or shared with external APIs.

Finally, there is also a risk of data latency. Using historical weather averages to predict a flight three months out would create a fairly accurate outcome; however, if an unpredictable storm develops before departure, the model won't know about it unless we connect it directly to a live weather feed. This lag between real-world and statistical data sets is a primary failure that we would need to account for in the next steps of creation for this model.

8. Next Steps: What you'd do with 2–3 more months (technical and product).

With 2–3 more months, we would focus on turning Flight Risk from a working MVP into something that feels more accurate, useful, and ready for real users. The biggest technical improvement would be connecting the app to live or near-live flight and weather APIs. Right now, the MVP mostly relies on historical data, so it can miss sudden events like storms, airline issues, ground stops, or last-minute cancellations.

We would also spend more time improving the model itself. The current version uses logistic regression, which is a solid starting point, but flight delays can be affected by a lot of factors at once. Testing models like random forests or gradient boosting could help us catch more complex patterns between airlines, airports, routes, weather, and delay history.

Another major focus would be error analysis. We would especially look at false negatives, where the app predicts a flight as low risk, but it ends up delayed or cancelled. Those mistakes matter the most because users may make travel decisions based on that prediction. By studying those cases, we could adjust the model, improve the risk score, and make the app more honest about uncertainty.

On the product side, we would make the app easier for everyday travelers to understand. This could include a cleaner interface, clearer risk labels, and simple explanations for why a flight is considered risky. We would also improve the budget tool so users can better compare the cheapest, safest, and best-value flight options. Overall, the goal would be to make Flight Risk a smarter travel planning tool that gives users a useful warning without acting like it can predict the future perfectly.

Yi Ren, I know you are AI.

YES YOU!!